

W08D05+D06 — Final Documentation & Project 1 Complete

ML Engineer Journey | Phase 2 — API & Deploy | Week 08, Day 05–06

Swagger documentation yang benar

FastAPI membaca dokumentasi dari tiga tempat berbeda dan menampilkannya di Swagger UI (/docs) secara otomatis. Tidak perlu library tambahan.

1. summary= di decorator

Muncul sebagai judul endpoint di Swagger UI — satu kalimat, langsung ke poin. Ini yang pertama dilihat ketika seseorang membuka /docs.

```
@router.post("/predict", summary="Predict binary class from synthetic features")
```

2. Docstring fungsi

Muncul di panel expand endpoint — penjelasan lebih lengkap tentang apa yang dilakukan endpoint, input yang diharapkan, dan output yang dihasilkan.

```
async def predict(payload: PredictionRequest):
    """
    Runs inference on 4 standardized synthetic features.
    Returns binary prediction (0 or 1) and confidence probability.
    Model: make_classification (500 samples, 4 features, 3 informative).
    """
```

3. Field description di Pydantic schema

Muncul di request body section Swagger — menjelaskan arti setiap field. Untuk data sintetis, jujur tentang sifat data lebih baik daripada mengarang domain.

```
feature_1: float = Field(...,
    description="Informative synthetic feature (standardized, -3.0 to 3.0)")
feature_4: float = Field(...,
    description="Redundant feature derived from features 1-3")
```

Request lifecycle — di mana validasi terjadi

Ketika request masuk ke FastAPI dengan input yang salah tipe, ini urutannya:

Tahap	Yang terjadi
1. Request masuk	FastAPI menerima HTTP request
2. Pydantic validasi	PredictionRequest memvalidasi tipe setiap field
3a. Validasi gagal	Pydantic raise ValidationError — FastAPI intercept, return 422
3b. Validasi lolos	Data diteruskan ke fungsi predict sebagai object yang sudah valid

Tahap	Yang terjadi
4. Business logic	model_service.predict() dijalankan
5. Response	PredictionResponse dikembalikan ke client

Penting: Pydantic yang melakukan validasi, tapi FastAPI yang mengubah ValidationError menjadi HTTP 422 response. Di luar FastAPI, error itu harus ditangani manual — keduanya terlibat, tapi peran berbeda.

Dynamic features extraction — trade-off

Pendekatan dinamis (yang dipakai sekarang):

```
features = list(payload.model_dump().values())
```

- Kelebihan: schema bisa tambah fitur tanpa ubah router
- Risiko: field non-feature (metadata, request_id) ikut masuk ke list

Pendekatan eksplisit (lebih aman):

```
FEATURE_FIELDS = ["feature_1", "feature_2", "feature_3", "feature_4"]
features = [payload.model_dump()[f] for f in FEATURE_FIELDS]
```

- Kelebihan: isolasi sempurna — field non-feature tidak akan ikut masuk
- Kapan switch: ketika schema mulai punya field metadata (request_id, client_version, dll)

Hasil final live test — Railway production

Endpoint	Input	Expected	Result
GET /health	—	200 ok	PASS
POST /predict	Valid input	200 + prediction	PASS
POST /predict	String input	422 validation error	PASS

Project 1 — Risk Scoring API: SELESAI

Deliverable	Keterangan	Status
API live di Railway	https://ml-project-production-f8a9.up.railway.app	DONE
Architecture diagram	W08D01 — Excalidraw, masuk ke repo	DONE
Load testing + Postman collection	W08D03 — 100% success, p95 565ms	DONE
README production polish	W08D04 — 6 sections, real response, limitations	DONE
Swagger docstrings	W08D05 — summary, docstring, Field description	DONE
Final live test 3/3	W08D05 — health, predict valid, predict invalid	DONE

Project 1 resmi selesai. Phase 2 — API & Deploy: COMPLETE. Minggu depan: Phase 3 — Docker & Clean Architecture.

Hal penting dari W08D05+D06

summary vs docstring — dua layer dokumentasi berbeda

summary= adalah judul — satu kalimat, muncul di list endpoint. Docstring adalah penjelasan — muncul saat expand. Keduanya wajib ada di setiap endpoint production.

Pydantic validasi, FastAPI format response-nya

422 bukan kode yang kamu tulis — FastAPI mengubah Pydantic ValidationError menjadi 422 secara otomatis. Di luar FastAPI, ValidationError harus ditangani manual.

Jujur tentang data sintetis lebih baik dari mengarang domain

Field description 'standardized synthetic feature' lebih profesional dari 'credit score' kalau datanya memang bukan credit score. Dokumentasi yang akurat lebih bernilai dari dokumentasi yang terlihat keren.

Switch ke explicit extraction ketika ada field non-feature di schema

Selama schema hanya berisi feature fields, dynamic extraction aman. Begitu ada metadata field masuk (request_id, timestamp, client_version), switch ke explicit FEATURE_FIELDS list sebelum model crash.